

Algorithm Analysis Lab

Compiled Code & Output Results (Simulated IDE View)

Experiment 1: Linear Search (Iterative)

exp1_linear_search.py - Visual Studio Code

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return f"Key found at index {i}"
    return "Key not found"

arr = [10, 25, 30, 45, 50]
print(linear_search(arr, 30))
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp1_linear_search.py
Key found at index 2
```

Experiment 2: Binary Search (Recursive)

exp2_binary_search.py - Visual Studio Code

```
def binary_search(arr, low, high, key):
    if high >= low:
        mid = (high + low) // 2
        if arr[mid] == key:
            return f"Key found at index {mid}"
        elif arr[mid] > key:
            return binary_search(arr, low, mid - 1, key)
        else:
            return binary_search(arr, mid + 1, high, key)
    return "Key not found"

arr = [5, 10, 15, 20, 25]
print(binary_search(arr, 0, len(arr)-1, 20))
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp2_binary_search.py
Key found at index 3
```

Experiment 3: Factorial (Iterative vs Recursive)

```
exp3_factorial.py - Visual Studio Code

def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)

print(factorial_recursive(5))

TERMINAL

PS C:\Users\Student\Lab> python exp3_factorial.py
120
```

Experiment 4: Fibonacci Series

```
exp4_fibonacci.py - Visual Studio Code

def fibonacci(n):
    fib_series = [0, 1]
    for i in range(2, n):
        fib_series.append(fib_series[i-1] + fib_series[i-2])
    return fib_series[:n]

print(fibonacci(6))

TERMINAL

PS C:\Users\Student\Lab> python exp4_fibonacci.py
[0, 1, 1, 2, 3, 5]
```

Experiment 5: Matrix Multiplication

exp5_matrix_mult.py - Visual Studio Code

```
def multiply_matrices(A, B):  
    result = [[0, 0], [0, 0]]  
    for i in range(len(A)):  
        for j in range(len(B[0])):  
            for k in range(len(B)):  
                result[i][j] += A[i][k] * B[k][j]  
    return result
```

```
A = [[1, 2], [3, 4]]  
B = [[5, 6], [7, 8]]  
print(multiply_matrices(A, B))
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp5_matrix_mult.py  
[[19, 22], [43, 50]]
```

Experiment 6: Merge Sort

exp6_merge_sort.py - Visual Studio Code

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L, R = arr[:mid], arr[mid:]
        merge_sort(L)
        merge_sort(R)
        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1
        while i < len(L):
            arr[k] = L[i]
            i += 1; k += 1
        while j < len(R):
            arr[k] = R[j]
            j += 1; k += 1
    return arr

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp6_merge_sort.py
[3, 9, 10, 27, 38, 43, 82]
```

Experiment 7: Quick Sort

```
exp7_quick_sort.py - Visual Studio Code

def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

print(quick_sort([10, 7, 8, 9, 1, 5]))
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp7_quick_sort.py
[1, 5, 7, 8, 9, 10]
```

Experiment 8: Tower of Hanoi

```
exp8_tower_hanoi.py - Visual Studio Code

def tower_of_hanoi(n, source, dest, aux):
    if n == 1:
        print(f"Move disk 1 from {source} to {dest}")
        return
    tower_of_hanoi(n-1, source, aux, dest)
    print(f"Move disk {n} from {source} to {dest}")
    tower_of_hanoi(n-1, aux, dest, source)

tower_of_hanoi(3, 'A', 'C', 'B')
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp8_tower_hanoi.py
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Experiment 9: GCD (Euclidean Algorithm)

```
exp9_gcd.py - Visual Studio Code

def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

print(gcd(48, 18))

TERMINAL

PS C:\Users\Student\Lab> python exp9_gcd.py
6
```

Experiment 10: Insertion Sort

```
exp10_insertion_sort.py - Visual Studio Code

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertion_sort([12, 11, 13, 5, 6]))

TERMINAL

PS C:\Users\Student\Lab> python exp10_insertion_sort.py
[5, 6, 11, 12, 13]
```

Experiment 11: Heap Sort

```
exp11_heap_sort.py - Visual Studio Code

import heapq

def heap_sort(arr):
    heapq.heapify(arr)
    return [heapq.heappop(arr) for _ in range(len(arr))]

print(heap_sort([4, 10, 3, 5, 1]))
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp11_heap_sort.py
[1, 3, 4, 5, 10]
```

Experiment 12: Counting Inversions

```
exp12_count_inversions.py - Visual Studio Code

def count_inversions(arr):
    inv_count = 0
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] > arr[j]:
                inv_count += 1
    return inv_count

print(count_inversions([2, 4, 1, 3, 5]))
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp12_count_inversions.py
3
```

Experiment 13: Maximum Subarray Sum

```
exp13_max_subarray.py - Visual Studio Code

def max_subarray_sum(arr):
    # Kadane's Algorithm
    max_so_far = float('-inf')
    curr_max = 0
    for i in range(len(arr)):
        curr_max += arr[i]
        if max_so_far < curr_max:
            max_so_far = curr_max
        if curr_max < 0:
            curr_max = 0
    return max_so_far

print(max_subarray_sum([-2, 3, 4, 1, 2, 1, 5, -3]))
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp13_max_subarray.py
16
```

Experiment 14: Exponentiation

```
exp14_exponentiation.py - Visual Studio Code

def power(x, n):
    if n == 0:
        return 1
    temp = power(x, n // 2)
    if n % 2 == 0:
        return temp * temp
    else:
        return x * temp * temp

print(power(2, 10))
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp14_exponentiation.py
1024
```


Experiment 15: Closest Pair of Points

```
exp15_closest_pair.py - Visual Studio Code

import math

def closest_pair(points):
    min_dist = float('inf')
    pair = None
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            p1, p2 = points[i], points[j]
            dist = math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
            if dist < min_dist:
                min_dist = dist
                pair = (p1, p2)
    return pair, min_dist

pts = [(1,2), (4,5), (7,8), (3,1)]
pair, dist = closest_pair(pts)
print(f"Closest pair {pair[0]}-{pair[1]}, Distance =  $\sqrt{{dist**2:.0f}}$ ")
```

TERMINAL

```
PS C:\Users\Student\Lab> python exp15_closest_pair.py
Closest pair (1, 2)-(3, 1), Distance =  $\sqrt{5}$ 
```